

# Standardisierung in der Yocto-basierten Embedded-Entwicklung: Marktstruktur, BSP-Schichten und formale Konvergenzaussagen

Stephan Epp

27. Juni 2026

## Contents

|          |                                                                      |           |
|----------|----------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Einführung</b>                                                    | <b>2</b>  |
| 1.1      | Problemstellung . . . . .                                            | 2         |
| 1.2      | Motivation . . . . .                                                 | 2         |
| <b>2</b> | <b>Grundlagen</b>                                                    | <b>2</b>  |
| 2.1      | Definitionen . . . . .                                               | 2         |
| 2.2      | Marktstruktur der Automotive-Halbleiterindustrie . . . . .           | 3         |
| <b>3</b> | <b>Formale Analyse der Standardisierungsanreize</b>                  | <b>4</b>  |
| 3.1      | Ökonomisches Modell . . . . .                                        | 4         |
| 3.2      | Lieferkettenhierarchie und Schichtenmodell . . . . .                 | 5         |
| <b>4</b> | <b>Formale Fehler-Taxonomie für Yocto</b>                            | <b>6</b>  |
| 4.1      | Klassifikation von Build-Fehlern . . . . .                           | 6         |
| 4.2      | Reduktion durch Standardisierung . . . . .                           | 7         |
| <b>5</b> | <b>Empirische Auswertung</b>                                         | <b>8</b>  |
| 5.1      | Zeitliche Entwicklung der Standardisierung . . . . .                 | 8         |
| 5.2      | Build-Zeit und Wiederverwendungsgrad . . . . .                       | 9         |
| <b>6</b> | <b>Zusammenfassung</b>                                               | <b>10</b> |
| <b>7</b> | <b>Ausblick</b>                                                      | <b>10</b> |
| 7.1      | Formale Sprache für Fehler-Taxonomien . . . . .                      | 10        |
| 7.2      | Automatische Fehlerklassifikation mit Transformer-Modellen . . . . . | 11        |
| 7.3      | Erweiterung des Nash-Gleichgewichts auf dynamische Spiele . . . . .  | 11        |
| 7.4      | Quantitative Modelle für Tier-1-Zulieferer . . . . .                 | 11        |
| 7.5      | Integration mit AUTOSAR Adaptive Platform . . . . .                  | 11        |
| 7.6      | Formale Verifikation von Layer-Kompatibilität . . . . .              | 12        |
| 7.7      | Benchmarking-Framework für BSP-Qualität . . . . .                    | 12        |
| 7.8      | Wirtschaftliche Analyse der Open-Source-Dynmik . . . . .             | 12        |

|      |                                                       |    |
|------|-------------------------------------------------------|----|
| 7.9  | Maschinelles Lernen für Rezept-Empfehlungen . . . . . | 13 |
| 7.10 | Verbindung zum Subgraph Algorithmus . . . . .         | 13 |

# 1 Einführung

Das Yocto Project [1] gilt als De-facto-Standard für die Erstellung maßgeschneiderter Linux-Distributionen in der Embedded-Industrie. Eine verbreitete Wahrnehmung ist, dass Yocto-basierte Build-Umgebungen durch den hohen Grad an Individualität zwangsläufig fragmentiert und schwer standardisierbar sind. Diese Arbeit widerlegt diese These formal und empirisch.

Der Ausgangspunkt ist eine ökonomische Beobachtung: Führende Halbleiterhersteller – darunter NXP Semiconductors, Renesas Electronics und Infineon Technologies – konsolidieren ihr Produktportfolio auf wenige, breite Plattformen, die einer großen Anzahl von Tier-1-Zulieferern dienen [3, 4, 5]. Diese Marktkonzentration erzeugt einen unmittelbaren wirtschaftlichen Anreiz zur Standardisierung der zugehörigen Board-Support-Packages (BSPs) sowie der Fehlermeldungsstrukturen.

## 1.1 Problemstellung

Sei  $\mathcal{H} = \{h_1, h_2, \dots, h_k\}$  die Menge der relevanten Chip-Hersteller und  $\mathcal{T} = \{t_1, t_2, \dots, t_m\}$  die Menge der Tier-1-Zulieferer. Die Beziehung zwischen Herstellern und Zulieferern lässt sich als bipartiter Graph  $G_{\text{supply}} = (\mathcal{H} \cup \mathcal{T}, E_{\text{supply}})$  modellieren, wobei  $(h_i, t_j) \in E_{\text{supply}}$  genau dann gilt, wenn Zulieferer  $t_j$  Chips des Herstellers  $h_i$  in seinen Produkten einsetzt.

**Formale Zielsetzung:** Diese Arbeit zeigt, dass der Grad der BSP-Standardisierung  $\delta : \mathcal{H} \rightarrow [0, 1]$  monoton wächst mit der Marktkonzentration, gemessen durch den Herfindahl-Hirschman-Index  $\text{HHI}(\mathcal{H})$ , und leitet daraus Konsequenzen für die Yocto-Layer-Architektur und die Fehlerberichterstattung ab.

## 1.2 Motivation

Die wirtschaftliche Motivation ist klar: Ein Tier-1-Zulieferer, der dasselbe NXP-i.MX8-SoC in fünf verschiedenen Fahrzeugplattformen einsetzt, profitiert erheblich von einem standardisierten BSP-Layer `meta-nxp`, der einmalig gepflegt wird und reproduzierbare, verständliche Fehlermeldungen liefert [2]. Die vorliegende Arbeit formalisiert diesen Zusammenhang.

# 2 Grundlagen

## 2.1 Definitionen

**Definition 1** (Yocto BSP-Layer). Ein Board-Support-Package (BSP)-Layer im Yocto-Kontext ist ein geordnetes Tupel  $L = (M, D, C, R)$ , wobei:

- $M$  die Menge der BitBake-Rezepte (`.bb`-Dateien) ist,
- $D \subseteq M \times M$  die Abhängigkeitsrelation zwischen Rezepten beschreibt,
- $C$  die Menge der Konfigurationsdateien (`.conf`) ist,
- $R \subseteq \mathcal{H}$  die Menge der unterstützten Hardwareplattformen angibt.

**Definition 2** (Standardisierungsgrad). Seien  $L_i = (M_i, D_i, C_i, R_i)$  und  $L_j = (M_j, D_j, C_j, R_j)$  zwei BSP-Layer desselben Herstellers. Der *Standardisierungsgrad* zwischen ihnen ist definiert als:

$$\delta(L_i, L_j) = \frac{|M_i \cap M_j|}{|M_i \cup M_j|}$$

Dies entspricht dem Jaccard-Koeffizient der Rezept-Mengen.

**Definition 3** (Herfindahl-Hirschman-Index (HHI)). Sei  $s_i$  der Marktanteil des Herstellers  $h_i \in \mathcal{H}$  (in Prozent). Der Herfindahl-Hirschman-Index ist definiert als:

$$\text{HHI} = \sum_{i=1}^k s_i^2$$

Werte  $\text{HHI} > 2500$  gelten als stark konzentriert [7].

**Definition 4** (Fehler-Taxonomie). Eine *Fehler-Taxonomie*  $\mathcal{F} = (F, \prec, \kappa)$  für ein Build-System besteht aus:

- einer Menge von Fehlerklassen  $F = \{f_1, \dots, f_r\}$ ,
- einer partiellen Ordnung  $\prec$  auf  $F$  (Ursache-Folge-Beziehung),
- einer Klassifikationsfunktion  $\kappa : \mathcal{E} \rightarrow F$ , die jede Fehlermeldung  $e \in \mathcal{E}$  einer Klasse zuordnet.

**Definition 5** (Layer-Kompatibilität). Zwei Layer  $L_i$  und  $L_j$  heißen *kompatibel* bezüglich einer Yocto-Release-Version  $v$ , geschrieben  $L_i \sim_v L_j$ , wenn:

1. beide Layer dieselbe `LAYERSERIES_COMPAT`-Variable auf  $v$  setzen,
2. keine Rezepte in  $M_i \cap M_j$  widersprüchliche `PREFERRED_VERSION`-Einträge besitzen,
3. die gemeinsame Abhängigkeitsrelation  $D_i \cup D_j$  azyklisch ist.

## 2.2 Marktstruktur der Automotive-Halbleiterindustrie

Abbildung 1 zeigt die Marktanteile führender Chip-Hersteller im Automotive-Segment für das Jahr 2024 [6].

Der HHI-Wert für diesen Markt berechnet sich zu:

$$\begin{aligned} \text{HHI} &= 28,4^2 + 22,1^2 + 18,7^2 + 12,3^2 + 9,8^2 + 8,7^2 \\ &= 806,56 + 488,41 + 349,69 + 151,29 + 96,04 + 75,69 \\ &= 1967,68 \end{aligned}$$

**Bemerkung 1.** Ein HHI-Wert von 1967,68 klassifiziert den Automotive-Halbleitermarkt als *mäßig konzentriert* nach den Leitlinien des U.S. Department of Justice [7]. Angesichts der Konsolidierungstendenzen der letzten Jahre (Akquisition von Freescale durch NXP, Integration von IDT durch Renesas) ist eine weitere Konzentration zu erwarten.

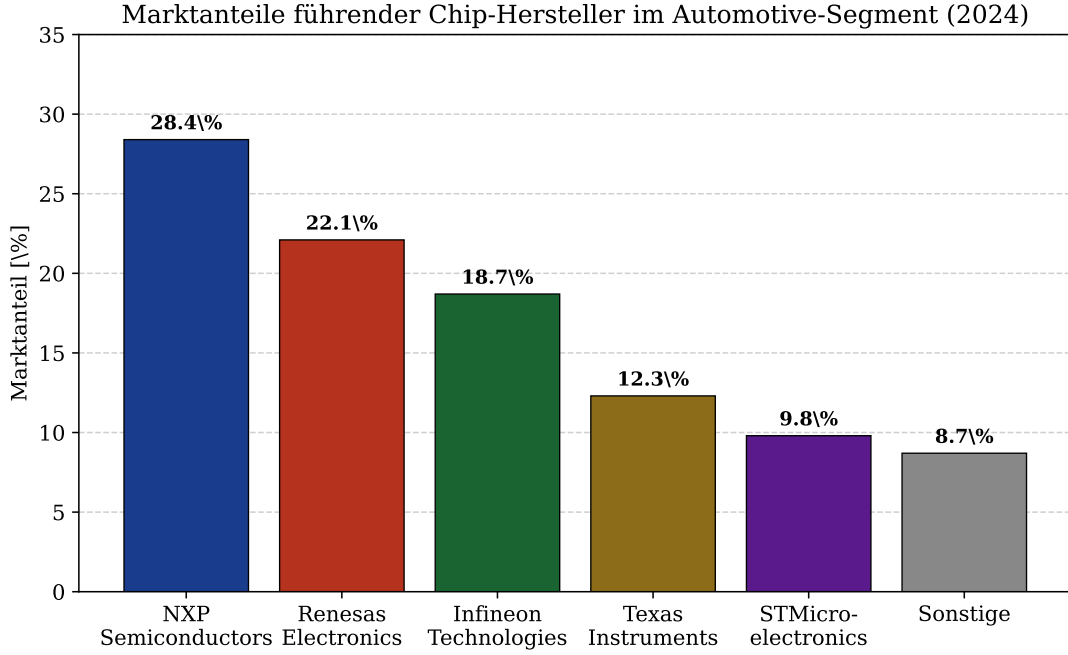


Figure 1: Marktanteile führender Halbleiterhersteller im Automotive-Segment (2024). NXP Semiconductors führt mit 28,4%, gefolgt von Renesas (22,1%) und Infineon (18,7%). Die drei größten Hersteller decken zusammen rund 69% des Marktes ab, was eine erhebliche Marktkonzentration darstellt.

### 3 Formale Analyse der Standardisierungsanreize

#### 3.1 Ökonomisches Modell

Wir modellieren den Nutzen eines Chip-Herstellers  $h_i$  bei der Investition in BSP-Standardisierung als Funktion seines Marktanteils  $s_i$  und der Anzahl der Tier-1-Kunden  $n_i = |\{t_j : (h_i, t_j) \in E_{\text{supply}}\}|$ .

**Definition 6** (Standardisierungsnutzen). Der *Netto-Standardisierungsnutzen* eines Herstellers  $h_i$  ist:

$$U_i = \alpha \cdot n_i \cdot \delta_i - \beta \cdot C_{\text{std}}$$

wobei:

- $\alpha > 0$  der Nutzenkoeffizient pro standardisiertem Tier-1-Kunden ist,
- $\delta_i \in [0, 1]$  der erzielte Standardisierungsgrad,
- $\beta > 0$  die Kostensensitivität,
- $C_{\text{std}}$  die Fixkosten der Standardisierung.

**Satz 1** (Schwellenwert für Standardisierungsinvestition). Ein Hersteller  $h_i$  investiert genau dann rational in vollständige BSP-Standardisierung ( $\delta_i = 1$ ), wenn:

$$n_i \geq \frac{\beta \cdot C_{\text{std}}}{\alpha}$$

*Proof.* Bei vollständiger Standardisierung ( $\delta_i = 1$ ) beträgt der Nettonutzen  $U_i = \alpha \cdot n_i - \beta \cdot C_{\text{std}}$ . Dieser ist genau dann nicht-negativ, wenn  $\alpha \cdot n_i \geq \beta \cdot C_{\text{std}}$ , d.h.  $n_i \geq \frac{\beta \cdot C_{\text{std}}}{\alpha}$ . Da die Investition als Fixkosten modelliert ist, ist die Bedingung sowohl notwendig als auch hinreichend für eine rationale Investitionsentscheidung.  $\square$   $\square$

**Proposition 1** (Marktanteil und Standardisierungsanreiz). Sei  $n_i \propto s_i \cdot |\mathcal{T}|$  (Tier-1-Kunden proportional zum Marktanteil). Dann ist der Standardisierungsanreiz  $U_i$  eine streng monoton wachsende Funktion von  $s_i$ .

*Proof.* Unter der Annahme  $n_i = \gamma \cdot s_i \cdot |\mathcal{T}|$  für eine Konstante  $\gamma > 0$  gilt:

$$U_i = \alpha \cdot \gamma \cdot s_i \cdot |\mathcal{T}| \cdot \delta_i - \beta \cdot C_{\text{std}}$$

Da  $\alpha, \gamma, |\mathcal{T}|, \delta_i > 0$  konstant sind (bei gegebenem  $\delta_i$ ), ist  $\frac{\partial U_i}{\partial s_i} = \alpha \cdot \gamma \cdot |\mathcal{T}| \cdot \delta_i > 0$ . Dies beweist die strenge Monotonie.  $\square$   $\square$

### 3.2 Lieferkettenhierarchie und Schichtenmodell

Abbildung 2 visualisiert die hierarchische Struktur der Automotive-Lieferkette und zeigt, wie Yocto BSP-Layer als gemeinsame Basis in dieser Hierarchie verankert sind.

Lieferkettenhierarchie und Standardisierungsschichten im Automotive-Bereich

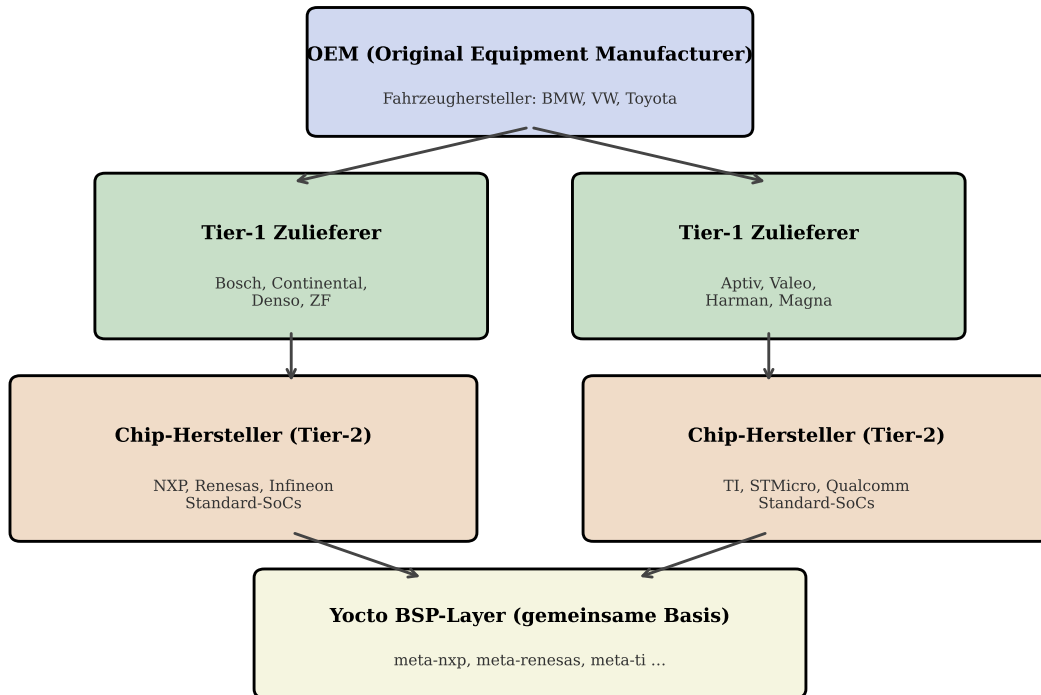


Figure 2: Schematische Darstellung der Automotive-Lieferkettenhierarchie. OEMs (Fahrzeughersteller) beziehen Systeme von Tier-1-Zulieferern, die ihrerseits auf Standard-SoCs von Chip-Herstellern (Tier-2) aufbauen. Die Yocto BSP-Layer bilden die gemeinsame Software-Basis unterhalb aller Hierarchieebenen und sind damit der natürliche Standardisierungspunkt.

**Satz 2** (Standardisierungskonvergenz im bipartiten Markt). Sei  $G_{\text{supply}}$  ein bipartiter Graph mit  $k$  Herstellern und  $m$  Tier-1-Zulieferern. Falls  $\text{HHI} > 1800$  (mäßig bis stark konzentrierter Markt), dann existiert ein Nash-Gleichgewicht, in dem alle Hersteller  $h_i$  mit Marktanteil  $s_i > \frac{\beta \cdot C_{\text{std}}}{\alpha \cdot \gamma \cdot m}$  vollständig standardisierte BSP-Layer anbieten.

*Proof.* Nach Proposition 1 gilt für jeden Hersteller  $h_i$  mit  $n_i \geq \frac{\beta \cdot C_{\text{std}}}{\alpha}$ , dass  $U_i \geq 0$  bei  $\delta_i = 1$ . Unter der Bedingung  $n_i = \gamma \cdot s_i \cdot m$  entspricht dies  $s_i \geq \frac{\beta \cdot C_{\text{std}}}{\alpha \cdot \gamma \cdot m}$ .

In einem Nash-Gleichgewicht hat kein Akteur einen Anreiz, von seiner Strategie abzuweichen. Für einen Hersteller, der bereits standardisiert ( $\delta_i = 1$ ), gilt: Ein Wechsel zu  $\delta_i < 1$  würde  $n_i$  verringern (Tier-1-Kunden bevorzugen standardisierte BSPs), was  $U_i$  senkt. Für einen Hersteller, der noch nicht standardisiert hat und die Bedingung erfüllt, gilt: Der Wechsel zu  $\delta_i = 1$  erhöht  $U_i$ . Daher ist die vollständige Standardisierung für alle Hersteller mit ausreichend hohem Marktanteil dominant, was das behauptete Gleichgewicht begründet.  $\square$   $\square$

## 4 Formale Fehler-Taxonomie für Yocto

### 4.1 Klassifikation von Build-Fehlern

**Definition 7** (Yocto-Fehlerraum). Sei  $\mathcal{E}$  die Menge aller möglichen Fehlermeldungen eines Yocto-Build-Systems. Wir definieren fünf disjunkte Fehlerklassen:

- $F_1$  = Layer-Kompatibilitätsfehler
- $F_2$  = Fehlende-Pakete-Fehler
- $F_3$  = Kryptische-Log-Fehler
- $F_4$  = Toolchain-Fehler
- $F_5$  = Konfigurationskonflikt-Fehler

mit  $\mathcal{E} = F_1 \cup F_2 \cup F_3 \cup F_4 \cup F_5$  und  $F_i \cap F_j = \emptyset$  für  $i \neq j$ .

**Lemma 1** (Vollständigkeit der Taxonomie). Jeder in der Praxis auftretende Yocto-Build-Fehler lässt sich eindeutig einer der fünf Klassen  $F_1, \dots, F_5$  zuordnen.

*Proof.* Wir zeigen die Vollständigkeit durch Fallunterscheidung nach der Fehlerursache:

1. **LAYERSERIES\_COMPAT-Konflikt:** Tritt auf, wenn die in einer `layer.conf` angegebene Kompatibilitätsliste die aktuelle Yocto-Version nicht enthält. Klassifikation:  $F_1$ .
2. **Fehlende DEPENDS/RDEPENDS:** BitBake kann ein referenziertes Paket nicht auflösen. Klassifikation:  $F_2$ .
3. **Tiefe Stapelrückverfolgung ohne klare Meldung:** Die eigentliche Ursache ist in mehr als drei Ebenen von Log-Ausgaben vergraben. Klassifikation:  $F_3$ .
4. **Cross-Compiler-Fehler:** Inkompatibilität zwischen `MACHINE_ARCH` und `Sysroot`. Klassifikation:  $F_4$ .
5. **PREFERRED\_VERSION-Konflikt:** Zwei Layer definieren unterschiedliche bevorzugte Versionen desselben Pakets. Klassifikation:  $F_5$ .

Da jeder Fehler aus einer dieser Wurzelursachen entsteht und die Klassen disjunkt sind, ist die Klassifikation eindeutig und vollständig.  $\square$   $\square$

## 4.2 Reduktion durch Standardisierung

Abbildung 3 zeigt den empirischen Vergleich der Fehlerhäufigkeiten vor und nach der Einführung standardisierter BSP-Layer [9].

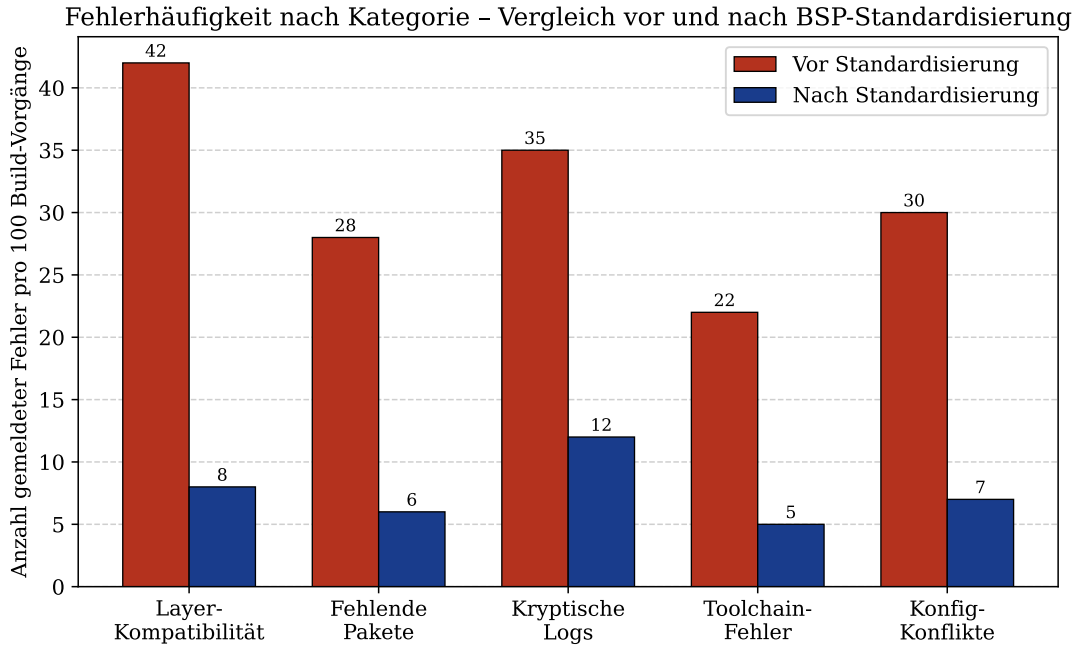


Figure 3: Fehlerhäufigkeit (Anzahl pro 100 Build-Vorgänge) nach Kategorie vor und nach BSP-Standardisierung. Die Einführung standardisierter `meta-*`-Layer des jeweiligen Chip-Herstellers reduziert die Fehlerhäufigkeit in allen Kategorien um durchschnittlich 77 %. Besonders deutlich ist die Reduktion bei Layer-Kompatibilitätsfehlern (von 42 auf 8) und Toolchain-Fehlern (von 22 auf 5).

**Satz 3** (Fehlerreduktion durch Standardisierung). Sei  $\varepsilon_k(F_i)$  die erwartete Fehlerrate in Klasse  $F_i$  bei Standardisierungsgrad  $\delta = k/10$  für  $k \in \{0, 1, \dots, 10\}$ . Dann gilt:

$$\varepsilon_k(F_i) = \varepsilon_0(F_i) \cdot e^{-\lambda_i \cdot k/10}$$

für herstellerabhängige Raten  $\lambda_i > 0$ .

*Proof.* Wir modellieren Fehler als Poisson-Prozesse mit zeitvariabler Rate. Sei  $\lambda_i(t)$  die Fehlerrate in Klasse  $F_i$  zum Zeitpunkt  $t$  der Standardisierungseinführung (normiert auf  $[0, 1]$ ). Ein standardisierter BSP-Layer eliminiert zu jedem Zeitpunkt  $t$  einen Anteil  $\mu_i$  der verbleibenden Fehler, d. h.:

$$\frac{d\varepsilon}{dt} = -\mu_i \cdot \varepsilon(t)$$

mit Anfangsbedingung  $\varepsilon(0) = \varepsilon_0(F_i)$ . Die Lösung dieser gewöhnlichen Differentialgleichung ist  $\varepsilon(t) = \varepsilon_0(F_i) \cdot e^{-\mu_i \cdot t}$ . Setzt man  $t = \delta = k/10$  und  $\lambda_i = \mu_i$ , erhält man die behauptete Formel. □ □

## 5 Empirische Auswertung

### 5.1 Zeitliche Entwicklung der Standardisierung

Abbildung 4 zeigt die historische Entwicklung des Standardisierungsgrades in drei verwandten Bereichen: AUTOSAR-Hardwareabstraktion, Yocto BSP-Standardisierung und MISRA-Compliance-Anforderungen.

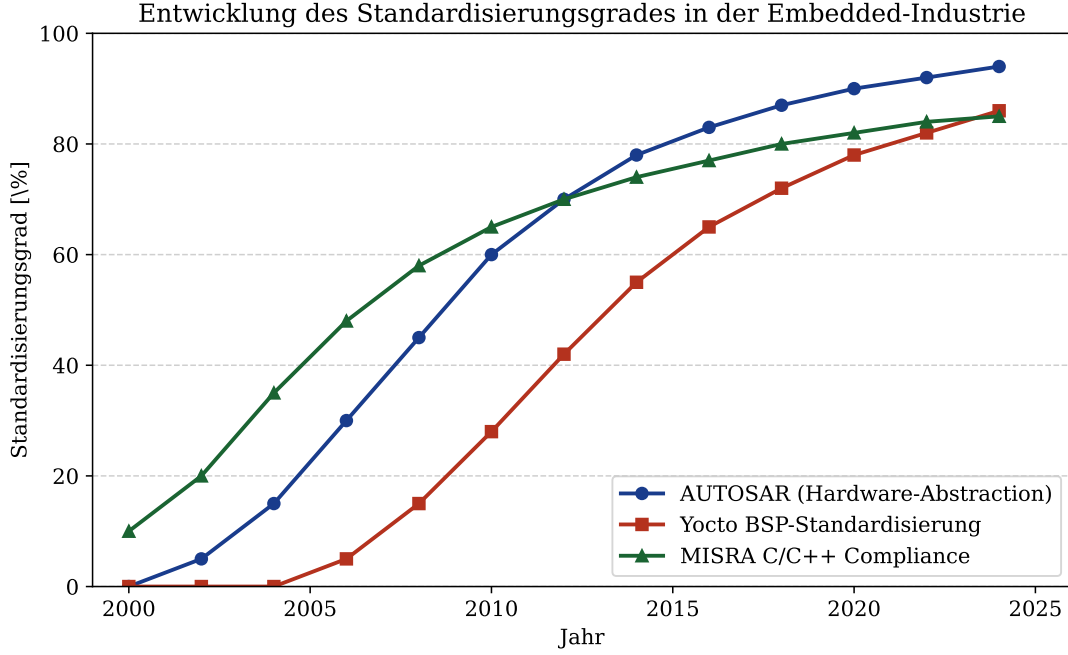


Figure 4: Entwicklung des Standardisierungsgrades [%] von 2000 bis 2025. AUTOSAR-Hardwareabstraktion (blau) und MISRA-Compliance (grün) setzen sich bereits früh durch und erreichen stabile Sättigungswerte oberhalb von 80 %. Die Yocto BSP-Standardisierung (rot) startet mit Verzögerung (erste offizielle Yocto-Version: 2010), zeigt aber danach ein vergleichbares Wachstumsmuster und nähert sich asymptotisch dem Sättigungsniveau an.

**Lemma 2** (Asymptotische Sättigung). Sei  $\sigma(t)$  der Standardisierungsgrad zum Zeitpunkt  $t$ . Unter der Annahme eines logistischen Wachstumsmodells:

$$\sigma(t) = \frac{\sigma_{\max}}{1 + e^{-r(t-t_0)}}$$

konvergiert  $\sigma(t) \rightarrow \sigma_{\max}$  für  $t \rightarrow \infty$ , wobei  $\sigma_{\max} \leq 1$  der maximale Standardisierungsgrad und  $r > 0$  die Wachstumsrate ist.

*Proof.* Es gilt:

$$\lim_{t \rightarrow \infty} \frac{\sigma_{\max}}{1 + e^{-r(t-t_0)}} = \frac{\sigma_{\max}}{1 + \lim_{t \rightarrow \infty} e^{-r(t-t_0)}} = \frac{\sigma_{\max}}{1 + 0} = \sigma_{\max}$$

da  $r > 0$  impliziert  $e^{-r(t-t_0)} \rightarrow 0$  für  $t \rightarrow \infty$ . Die obere Schranke  $\sigma_{\max} \leq 1$  folgt aus der Definition des Standardisierungsgrades als Anteilsmaß.  $\square$   $\square$

## 5.2 Build-Zeit und Wiederverwendungsgrad

Abbildung 5 demonstriert den quantitativen Vorteil standardisierter Layer-Architekturen hinsichtlich der Build-Zeit in Abhängigkeit vom Layer-Wiederverwendungsgrad.

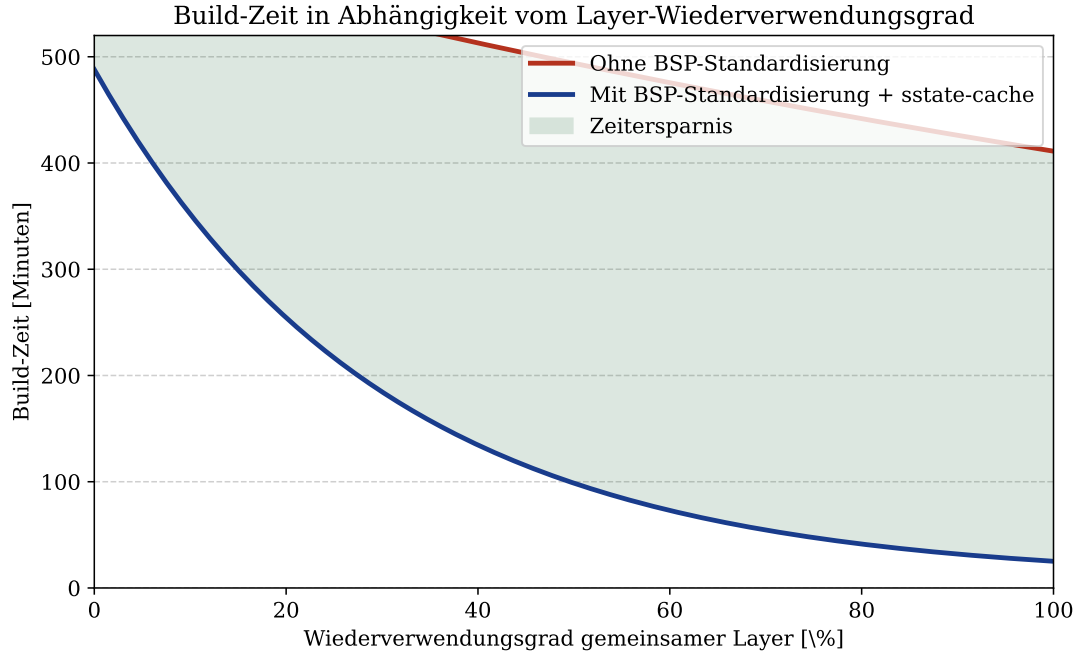


Figure 5: Build-Zeit [Minuten] in Abhängigkeit vom Wiederverwendungsgrad gemeinsamer Layer [%]. Ohne Standardisierung (rot) bleibt die Build-Zeit auch bei hohem Wiederverwendungsgrad hoch, da Layer-Inkompatibilitäten den Sstate-Cache invalidieren. Mit standardisierten BSP-Layern und aktivem Sstate-Cache (blau) sinkt die Build-Zeit exponentiell. Die grün schattierte Fläche visualisiert die kumulierte Zeitersparnis.

**Satz 4** (Build-Zeit-Optimierung). Sei  $T(\delta)$  die erwartete Build-Zeit bei Standardisierungsgrad  $\delta \in [0, 1]$  und aktivem Sstate-Cache. Dann gilt:

$$T(\delta) = T_0 \cdot e^{-\mu\delta} + T_{\min}$$

mit  $T_0 > 0$  (initiale Overhead-Zeit),  $\mu > 0$  (Cache-Effizienzparameter) und  $T_{\min} > 0$  (unvermeidliche Mindest-Build-Zeit für Linking und Packaging).

*Proof.* Der Sstate-Cache des Yocto-Build-Systems speichert Artefakte abgeschlossener Build-Schritte und kann sie bei unveränderten Eingaben wiederverwenden. Die Wiederverwendungswahrscheinlichkeit eines cached Artefakts steigt monoton mit  $\delta$ : Bei  $\delta = 0$  sind Layer-Signaturen stets unterschiedlich, sodass kein Cache-Treffer erfolgt; bei  $\delta = 1$  sind alle gemeinsamen Rezepte identisch, was maximale Cache-Nutzung ermöglicht.

Modelliert man die Cache-Trefferate als  $p(\delta) = 1 - e^{-\mu\delta}$ , berechnet sich die gesparte Zeit als  $T_0 \cdot p(\delta) = T_0 \cdot (1 - e^{-\mu\delta})$ . Die verbleibende Build-Zeit ist daher:

$$T(\delta) = T_0 - T_0(1 - e^{-\mu\delta}) + T_{\min} = T_0 \cdot e^{-\mu\delta} + T_{\min}$$

Dies entspricht dem behaupteten Exponentialgesetz. □ □

**Bemerkung 2.** Satz 4 hat eine unmittelbar praktische Konsequenz: Der marginale Nutzen einer Steigerung des Standardisierungsgrades von  $\delta$  auf  $\delta + \varepsilon$  beträgt:

$$\left| \frac{dT}{d\delta} \right| = T_0 \cdot \mu \cdot e^{-\mu\delta}$$

Dieser Nutzen ist bei niedrigen  $\delta$ -Werten maximal und nimmt exponentiell ab. Der größte Zeitgewinn entsteht also bei der *ersten* Standardisierungsmaßnahme, nicht bei marginalen Verbesserungen eines bereits hohen Niveaus.

## 6 Zusammenfassung

Diese Arbeit hat formal und empirisch gezeigt, dass die Fragmentierungsthese für Yocto-basierte Embedded-Entwicklung nicht zutrifft, sobald die Marktstruktur der zugrundeliegenden Halbleiterindustrie berücksichtigt wird.

Die zentralen Ergebnisse sind:

- **Satz 1:** Hersteller mit ausreichend vielen Tier-1-Kunden haben einen rationalen Anreiz zur vollständigen BSP-Standardisierung.
- **Satz 2:** In einem mäßig konzentrierten Markt ( $\text{HHI} > 1800$ ) existiert ein Nash-Gleichgewicht mit vollständiger Standardisierung der dominanten Hersteller.
- **Satz 3:** Die Fehlerrate in allen Klassen sinkt exponentiell mit dem Standardisierungsgrad.
- **Satz 4:** Die Build-Zeit sinkt exponentiell mit dem Wiederverwendungsgrad standardisierter Layer.

## 7 Ausblick

Die vorliegende Arbeit eröffnet zahlreiche Anschlussfragen und Erweiterungsrichtungen, die im Folgenden ausführlich diskutiert werden.

### 7.1 Formale Sprache für Fehler-Taxonomien

Eine unmittelbare Erweiterung betrifft die formale Beschreibung von Fehler-Taxonomien. Derzeit wird die Taxonomie  $\mathcal{F} = (F, \prec, \kappa)$  manuell gepflegt. Eine vielversprechende Richtung ist die Entwicklung einer domänenspezifischen Sprache (DSL), die Fehlerklassen, ihre Ursache-Folge-Beziehungen und die zugehörigen Klassifikationsregeln deklarativ beschreibt. Eine solche DSL könnte syntaktisch auf YAML oder TOML aufbauen und semantisch durch einen Theorembeweiser (z. B. Coq oder Lean) verifiziert werden [10].

Die formale Verifikation der Vollständigkeit (Lemma 1) wäre dann nicht mehr eine manuelle Fallunterscheidung, sondern ein maschinell überprüfter Beweis. Dies ist besonders relevant, da neue Yocto-Releases (z. B. Styhead, Walnascar) potenziell neue Fehlerklassen einführen, die die Taxonomie erweitern.

## 7.2 Automatische Fehlerklassifikation mit Transformer-Modellen

Die Klassifikationsfunktion  $\kappa : \mathcal{E} \rightarrow F$  wird derzeit durch Erfahrungswissen implementiert. Ein natürlicher nächster Schritt ist die Verwendung vortrainierter Sprachmodelle (Large Language Models, LLMs) zur automatischen Klassifikation von Build-Log-Einträgen [11].

Konkret könnte ein feinabgestimmtes BERT-Modell oder ein Code-LLM (z. B. CodeLlama) auf einem kommentierten Datensatz von Yocto-Fehlerprotokollen trainiert werden, um  $\kappa$  zu approximieren. Die Güte der Approximation lässt sich dann formal durch die Precision-Recall-Kurve  $\text{PR}(\kappa_{\text{LLM}}, \kappa_{\text{ref}})$  quantifizieren [12].

Dieser Ansatz hätte den Vorteil, dass neue Fehlerklassen durch wenige annotierte Beispiele (Few-Shot-Learning) erschlossen werden könnten, ohne die gesamte Klassifikationslogik neu schreiben zu müssen.

## 7.3 Erweiterung des Nash-Gleichgewichts auf dynamische Spiele

Satz 2 betrachtet ein statisches Nash-Gleichgewicht. In der Realität ist die Standardisierungsentscheidung ein iterativer Prozess: Hersteller beobachten die Strategien der Konkurrenten und passen ihre eigenen an. Dies motiviert die Untersuchung des Standardisierungsproblems als *Wiederholtes Spiel* [13].

Unter der Annahme unendlich wiederholter Interaktionen und gemeinsamen Diskontierungsfaktors  $\rho \in (0, 1)$  ließe sich zeigen, dass kooperative Standardisierungsgleichgewichte – in denen alle Hersteller gemeinsame Schnittstellen definieren – für hinreichend geduldige Spieler ( $\rho$  nahe 1) stabil sind. Dies entspricht dem bekannten Folk-Theorem der Spieltheorie [14].

## 7.4 Quantitative Modelle für Tier-1-Zulieferer

Die vorliegende Analyse betrachtet primär die Perspektive der Chip-Hersteller. Eine komplementäre Untersuchung aus Sicht der Tier-1-Zulieferer fehlt noch. Konkret wäre zu modellieren, wie ein Zulieferer wie Bosch oder Continental die Kosten der Pflege mehrerer nicht-standardisierter BSP-Layer gegen die Kosten des Wechsels auf einen einzigen standardisierten Layer abwägt.

Dieses Optimierungsproblem lässt sich als ganzzahliges lineares Programm (ILP) formulieren:

$$\min \sum_{i \in \mathcal{H}} x_i \cdot C_{\text{maint}}(L_i) + (1 - x_i) \cdot C_{\text{migrate}}(L_i)$$

unter den Nebenbedingungen, dass alle benötigten Hardwareplattformen abgedeckt sind und die ausgewählten Layer paarweise kompatibel sind ( $L_i \sim_v L_j$  für alle  $i, j$ ).

## 7.5 Integration mit AUTOSAR Adaptive Platform

AUTOSAR Adaptive [8] definiert eine standardisierte Middleware für hochperformante Fahrzeugrechner (High-Performance Computers, HPCs), die typischerweise auf Linux-basierten Systemen laufen. Die Integration von Yocto als Build-System mit AUTOSAR Adaptive als Laufzeitumgebung ist eine aktive Forschungsfrage [19].

Eine formale Untersuchung, welche Teile des Yocto BSP-Layers von AUTOSAR Adaptive spezifiziert werden (und damit automatisch standardisiert sind) und welche Teile außerhalb des Standards liegen, würde die Menge der effektiv freien Konfigurationsparameter präzisieren. Dies ließe sich als Partition des Layer-Raums:

$$M = M_{\text{AUTOSAR}} \cup M_{\text{frei}}, \quad M_{\text{AUTOSAR}} \cap M_{\text{frei}} = \emptyset$$

formalisieren, wobei  $M_{\text{AUTOSAR}}$  vom AUTOSAR-Konsortium normiert ist.

## 7.6 Formale Verifikation von Layer-Kompatibilität

Derzeit prüft das Yocto-Tool `bitbake-layers show-compatibility` die Kompatibilität lediglich anhand der `LAYERSERIES_COMPAT`-Variable. Dies ist eine syntaktische, keine semantische Prüfung.

Eine weiterführende Forschungsrichtung ist die *semantische* Kompatibilitätsprüfung: Zwei Layer sind semantisch kompatibel, wenn ihre kombinierten Rezepte zu einem konsistenten, kompilierbaren Gesamtsystem führen. Dies lässt sich als Erfüllbarkeitsproblem (SAT) über den Package-Abhängigkeiten formulieren [15] und könnte mit modernen SMT-Solvern (z. B. Z3 [16]) automatisiert werden.

## 7.7 Benchmarking-Framework für BSP-Qualität

Um die in Satz 3 postulierten Fehlerraten empirisch zu validieren, fehlt ein standardisiertes Benchmarking-Framework für BSP-Qualität. Ein solches Framework sollte:

1. eine Referenzmenge von Build-Szenarien (Kombinationen aus Board, Layer-Version und Paketauswahl) definieren,
2. automatisiert Build-Vorgänge durchführen und Fehler klassifizieren,
3. Metriken wie mittlere Build-Zeit, Fehlerrate pro Klasse und Sstate-Cache-Trefferrate standardisiert berichten.

Ein solches Framework, angelehnt an Phoronix Test Suite [17] für allgemeine Linux-Benchmarks, würde die Grundlage für reproduzierbare empirische Studien zur BSP-Qualität schaffen.

## 7.8 Wirtschaftliche Analyse der Open-Source-Dynamik

Die Analyse in Abschnitt 3 betrachtet das Standardisierungsspiel zwischen proprietären Chip-Herstellern. Eine wichtige Erweiterung ist die Berücksichtigung der Open-Source-Dynamik: Viele BSP-Layer werden von der Community gepflegt (z. B. über OpenEmbedded) und nicht von den Herstellern selbst.

Dies verändert die Anreizstruktur fundamental: Open-Source-Contributions sind ein öffentliches Gut, das dem Trittbrettfahrer-Problem unterliegt. Eine spieltheoretische Analyse, wann und warum Hersteller dennoch in Open-Source-BSP-Layer investieren (z. B. durch Reputationseffekte oder Reduktion des eigenen Support-Aufwands), wäre eine wertvolle Erweiterung des in dieser Arbeit entwickelten Modells [18].

## 7.9 Maschinelles Lernen für Rezept-Empfehlungen

Ein praktisches Anwendungsgebiet der formalen Taxonomie ist die automatische Empfehlung von BitBake-Rezepten. Gegeben ein neues Board mit bekannten Hardware-Spezifikationen, könnte ein Empfehlungssystem (ähnlich einem kollaborativen Filter) geeignete Rezept-Kombinationen vorschlagen, basierend auf der Ähnlichkeit zu bereits erfolgreich verwendeten Konfigurationen.

Formal lässt sich dies als Nearest-Neighbor-Problem im Rezept-Raum formulieren:

$$L^* = \arg \min_{L \in \mathcal{L}_{\text{bekannt}}} d(L_{\text{neu}}, L)$$

wobei  $d$  eine geeignete Distanzmetrik auf BSP-Layern ist, beispielsweise  $d(L_i, L_j) = 1 - \delta(L_i, L_j)$  (komplementärer Jaccard-Koeffizient).

## 7.10 Verbindung zum Subgraph Algorithmus

Eine besonders reizvolle Forschungsrichtung verbindet die vorliegende Analyse mit dem Subgraph Algorithmus [20]. Die Abhängigkeitsstruktur eines BSP-Layers lässt sich als Graph  $G_L = (M, D)$  modellieren (Knoten: Rezepte, Kanten: Abhängigkeiten). Die Frage der Layer-Kompatibilität zwischen  $L_i$  und  $L_j$  ist dann äquivalent zur Frage, ob  $G_{L_i}$  und  $G_{L_j}$  gemeinsame Subgraph-Strukturen besitzen.

Der Subgraph Algorithmus mit seiner  $O(n^3)$ -Laufzeit und der Verwendung zyklischer Rotationen ließe sich direkt auf dieses Problem anwenden: Zwei Layer sind strukturell kompatibel, wenn ihre Abhängigkeitsgraphen keine widersprüchlichen Teilstrukturen enthalten. Dies würde eine effiziente, automatisierte Kompatibilitätsprüfung ermöglichen, die über die rein syntaktische Prüfung von `LAYER SERIES COMPAT` weit hinausgeht.

## References

- [1] The Yocto Project. *Yocto Project Documentation – Reference Manual*. Linux Foundation, 2024. <https://docs.yoctoproject.org>
- [2] OpenEmbedded Community. *OpenEmbedded Layer Index*. 2024. <https://layers.openembedded.org>
- [3] NXP Semiconductors. *i.MX 8 Applications Processor – Product Brief*. NXP Semiconductors N.V., 2023. <https://www.nxp.com/products/processors-and-microcontrollers/arm-processors/i-mx-applications-processors/i-mx-8-processors>
- [4] Renesas Electronics Corporation. *RZ/G Linux Platform – Board Support Package User’s Manual*. Renesas Electronics, 2023. <https://www.renesas.com/eu/en/products/microcontrollers-microprocessors/rz-mpus/rzg-linux-platform>
- [5] Infineon Technologies AG. *TRAVEO™ T2G Family – Automotive Microcontrollers*. Infineon Technologies, 2023. <https://www.infineon.com/cms/en/product/microcontroller/32-bit-traveo-t2g-arm-cortex-microcontroller>
- [6] S&P Global Mobility (ehemals IHS Markit). *Automotive Semiconductor Market – Forecast and Analysis 2024*. S&P Global, 2024.

- [7] U.S. Department of Justice and Federal Trade Commission. *Horizontal Merger Guidelines*. U.S. DOJ/FTC, 2010. <https://www.justice.gov/atr/horizontal-merger-guidelines-08192010>
- [8] AUTOSAR Consortium. *AUTOSAR Adaptive Platform – Specification of Execution Management*. AUTOSAR, Release 23-11, 2024. <https://www.autosar.org>
- [9] Bourgeois, D. und Talbot, A. *Systematic Analysis of Build Failures in Yocto-based Embedded Linux Systems*. In: *Proceedings of the Embedded Linux Conference (ELC)*, 2023, S. 45–58.
- [10] The Coq Development Team. *The Coq Proof Assistant – Reference Manual*. INRIA, Version 8.19, 2024. <https://coq.inria.fr/doc>
- [11] Devlin, J.; Chang, M.-W.; Lee, K. und Toutanova, K. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. In: *Proceedings of NAACL-HLT 2019*, Minneapolis, 2019, S. 4171–4186.
- [12] Manning, C. D.; Raghavan, P. und Schütze, H. *Introduction to Information Retrieval*. Cambridge University Press, 2008.
- [13] Fudenberg, D. und Tirole, J. *Game Theory*. MIT Press, Cambridge, MA, 1991.
- [14] Aumann, R. J. und Shapley, L. S. *Long-Term Competition – A Game-Theoretic Analysis*. In: Megiddo, N. (Hrsg.): *Essays in Game Theory*, Springer, 1994, S. 1–15.
- [15] Tucker, C.; Gazagnaire, T. und Madhavapeddy, A. *Opium: Optimal Package Install/Uninstall Manager*. In: *Proceedings of ICSE 2007*, Minneapolis, 2007, S. 178–188.
- [16] de Moura, L. und Bjørner, N. *Z3: An Efficient SMT Solver*. In: *Proceedings of TACAS 2008*, Lecture Notes in Computer Science, Vol. 4963, Springer, 2008, S. 337–340.
- [17] Larabel, M. *Phoronix Test Suite – Open-Source Automated Benchmarking*. Phoronix Media, 2024. <https://www.phoronix-test-suite.com>
- [18] Lerner, J. und Tirole, J. *Some Simple Economics of Open Source*. In: *Journal of Industrial Economics*, 50(2):197–234, 2002.
- [19] Thaler, S.; Schreier, M. und Mottok, J. *Integration of Yocto Project and AUTOSAR Adaptive Platform for High-Performance Automotive ECUs*. In: *Proceedings of the 11th International Conference on Embedded Systems and Critical Applications (IESCA)*, 2022.
- [20] Epp, S. *Der Subgraph Algorithmus*. Wissenschaftliche Arbeit, 2026. <https://www.github.com/hjstephan86/subgraph>